# PW 01: Data Preparation

# 2- Data cleaning

- **Objective:** Gain practical skills in data preprocessing using Pandas. Learn how to detect, handle, and correct missing, inconsistent, or duplicate data to ensure dataset quality.

In [ ]:
```
import pandas as pd


# Replace 'your_excel_file.xlsx' with the path to your Excel file
excel_file = 'sales_data_1.xlsx'

# Load Excel data into a pandas DataFrame
df = pd.read_excel(excel_file)

# Now, you can work with the data in the 'df' DataFrame
# For example, you can print the first few rows of the DataFrame
df.head(100)
```

## Part 1 : Imputing Missing Values

### Objective

- Detect and handle missing values in a dataset.
- Use the `SimpleImputer` method from `scikit-learn` to fill missing data efficiently.
- Apply different imputation strategies for numeric and categorical variables.

---

### Step 1: Identify Missing Values

```
# Check for missing values in the entire DataFrame
missing_values = df.isnull().sum()

# Print only columns with missing values
print("Missing Values:")
print(missing_values[missing_values > 0])
```

**Explanation:**

- `df.isnull()` → returns True/False for each cell (True = missing).
- `.sum()` → counts missing values per column.
- `missing_values[missing_values > 0]` → filters columns that actually have missing data.

---

In [ ]:
```
missing_values = df.isnull().sum()
missing_values
```

In [ ]:
```
print("Missing Values:")
print(missing_values[missing_values > 0])
```

### Step 2: Install Required Library

```
pip install scikit-learn
```

- `scikit-learn` (or `sklearn`) → essential Python library for machine learning.
- Contains the `SimpleImputer` class used to replace missing values.

---

### Step 3: Import the SimpleImputer

```
from sklearn.impute import SimpleImputer
```

This module provides multiple strategies to replace missing data:

- **mean** → replaces with the average (numeric only)
- **median** → replaces with the median (recommended for numeric data with outliers)
- **most_frequent** → replaces with the most common value (for categorical data)
- **constant** → replaces with a fixed value (user-defined)

---

In [ ]:
```
from sklearn.impute import SimpleImputer
```

## Step 4: Impute Numeric Data (e.g., `order_value_EUR`)

```
target_column = 'order_value_EUR'

# Create and apply the SimpleImputer using the median strategy
imputer = SimpleImputer(strategy='median')
df[target_column] = imputer.fit_transform(df[[target_column]])
```

**Explanation:**

- `fit_transform()` → calculates the median and replaces missing values.
- Double brackets `[[ ]]` ensure a 2D format required by scikit-learn.
- After this operation, the numeric column no longer has missing data.

---

In [ ]:
```
target_column = 'order_value_EUR'

# Create a SimpleImputer instance to impute missing values with a strategy (e.g., 'mean', 'median',
'most_frequent')
# Other strategies include 'constant' to replace missing values with a constant value
imputer = SimpleImputer(strategy='median')

# Fit the imputer on the selected column and transform it to impute missing values
df[target_column] = imputer.fit_transform(df[[target_column]]).ravel()
# ravel(): flattens a multi-dimensional array into a one-dimensional array.
```

In [ ]:
```
df.isnull().sum() #verify
```

## Step 5: Impute Categorical Data (e.g., `device_type`)

```
target_column = 'device_type'

# Use the 'most_frequent' strategy for categorical variables
imputer = SimpleImputer(strategy='most_frequent')
df[target_column] = imputer.fit_transform(df[[target_column]])
```

**Explanation:**

- `most_frequent` replaces missing values with the most common category (e.g., "PC").
- Works best for non-numeric or text-based columns.

---

## Step 6: Verify Missing Values

```
df.isnull().sum()
```

Output should confirm **zero missing values** in the dataset:

```
country            0
order_value_EUR    0
cost               0
date               0
device_type        0
...
dtype: int64
```

---

In [ ]:
```
df.device_type
```

In [ ]:
```
target_column = 'device_type'

# Create a SimpleImputer instance to impute missing values with a strategy (e.g., 'mean', 'median',
'most_frequent')
# Other strategies include 'constant' to replace missing values with a constant value
imputer = SimpleImputer(strategy='most_frequent')


# Fit the imputer on the selected column and transform it to impute missing values
df[target_column] = imputer.fit_transform(df[[target_column]]).ravel()
```

In [ ]:
```
df.isnull().sum()
```

In [ ]:
```
df
```

### fill missing values with `fillna`

In [ ]:
```
# Create 3 empty rows with all columns set to NaN (except optional 'index')
import numpy as np
empty_rows = pd.DataFrame([{
     'country': np.nan,
     'order_value_EUR': np.nan,
     'cost': np.nan,
     'date': np.nan,
     'category': np.nan,
     'customer_name': np.nan,
     'sales_manager': np.nan,
     'device_type': np.nan,
     'order_id': np.nan,
} for i in range(3)])

# Append to the existing DataFrame
df = pd.concat([df, empty_rows], ignore_index=True)
```

In [ ]:
```
# Show the updated DataFrame
df.tail(5) # Show last 5 rows including the 3 new ones
```

In [ ]:
```
df ['order_value_EUR'] = df ['order_value_EUR'].fillna (df['order_value_EUR'].mean())   # fillna used here to replace NaN values with mean
```

In [ ]:
```
df
```

### fill missing values with `ffill` and `bfill`

In [ ]:
```
df = df.ffill() # forward fill
df = df.bfill() # backward fill
```

In [ ]:
```
df
```

In [ ]:
```
df[df['order_value_EUR'].isnull()]
```

### Handling NaN Values with Conditional Replacement (loc method)

In [ ]:
```
# String column: fill with constant value (e.g., "Unknown")
df.loc[df['country'].isnull(), 'country'] = 'Algeria'
```

In [ ]:
```
#  Custom condition: fill NaN based on another column
df.loc[(df['cost'].isnull()) & (df['country'] == 'UK'), 'cost'] = 250000
```

## Part 2 : Handling Non-Numeric and change types

In [ ]:
```
# Sample variable containing mixed data
mixed_data = df['cost']

# Create an empty list to store non-numeric values
non_numeric_values = []

# Iterate through the elements in the variable
for value in mixed_data:
     if isinstance(value, str) and not value.isnumeric():
         non_numeric_values.append(value)

# Print the non-numeric values
print("Non-Numeric Values:")
print(non_numeric_values)
```

In [ ]:
```
# Remove this value

# df = df[df['cost'] != 'XXX']

df = df[~df['cost'].isin(non_numeric_values)]
```

### Standardization of data types

In [ ]:
```
df['cost'] = df['cost'].astype(float) #convert type to float
df['date'] = pd.to_datetime(df['date'])
```

In [ ]:
```
df.dtypes
```

## Part 3 : Remove Duplicate values

In [ ]:
```
# Identify duplicate rows based on all columns
duplicates = df[df.duplicated()]

# Print duplicate rows
print("Duplicate Rows:")
duplicates
```

In [ ]:
```
# Remove duplicate rows, keeping the first occurrence
df_no_duplicates = df.drop_duplicates()

# Print the DataFrame without duplicates
print("DataFrame without Duplicates:")
df_no_duplicates
```

In [ ]:
```
# another exemple of remove duplicate
```

In [ ]:
```
import pandas as pd

# Create a DataFrame with duplicate rows (replace this with your actual data)
data = {
     'Name': ['Alice', 'Bob', 'Alice', 'Charlie', 'Bob'],
     'Age': [25, 30, 25, 35, 30],
     'Salary': [50000, 60000, 50000, 75000, 60000]
}

df1 = pd.DataFrame(data)

# Identify duplicate rows based on all columns
duplicates = df1[df1.duplicated()]

# Print duplicate rows
print("Duplicate Rows:")
print(duplicates)

# Remove duplicate rows, keeping the first occurrence
df_no_duplicates = df1.drop_duplicates()

# Print the DataFrame without duplicates
print("DataFrame without Duplicates:")
print(df_no_duplicates)
```

## Part 4 : String Clearning

In [ ]:
```
import pandas as pd

# Create a small messy dataset with two columns
data = {
     'city': [
         ' New-York ',
         'PARIS ',
         ' los-angeles ',
         'Rio-de-Janeiro',
         ' méxico-city ',
         'São Paulo',
         'Zürich ',
         'Montréal ',
         ' Bogotá ',
         'Cairo'
     ],
     'population': [8419600, 2148000, 3980000, 6748000, 9209944, 12330000, 434000, 1780000, 7744000, 9900000]
}


df2 = pd.DataFrame(data)
print("Before cleaning:\n", df2)

# Clean the city names
df2['city'] =df2['city'].str.strip().str.lower().str.replace('-', ' ')

print("\nAfter cleaning:\n", df2)
```

In [ ]:
```

```

In [ ]:
```
df
```

## Part 5 : Normalization and Standardization

## MinMaxScaler --- Normalization

Scales values to a fixed range, usually **[0, 1]**.

**Formula:**
$$ X' = \frac{X - X_{min}}{X_{max} - X_{min}} $$

**Example:**
If `X = [10, 20, 30]`
→ `X' = [0, 0.5, 1]`

---

## StandardScaler --- Standardization

Centers data around **mean = 0** and scales to **standard deviation = 1**.

**Formula:**
$$ Z = \frac{X - \mu}{\sigma} $$

**Example:**
If `X = [10, 20, 30]`, Mean = 20, Std = 8.16
→ `Z = [-1.22, 0, +1.22]`

In [ ]:
```
from sklearn.preprocessing import MinMaxScaler , StandardScaler

df['cost1'] = MinMaxScaler().fit_transform(df[['cost']])
df['cost2'] = StandardScaler().fit_transform(df[['cost']])
```

In [ ]:
```
df
```

In [ ]:
```

```